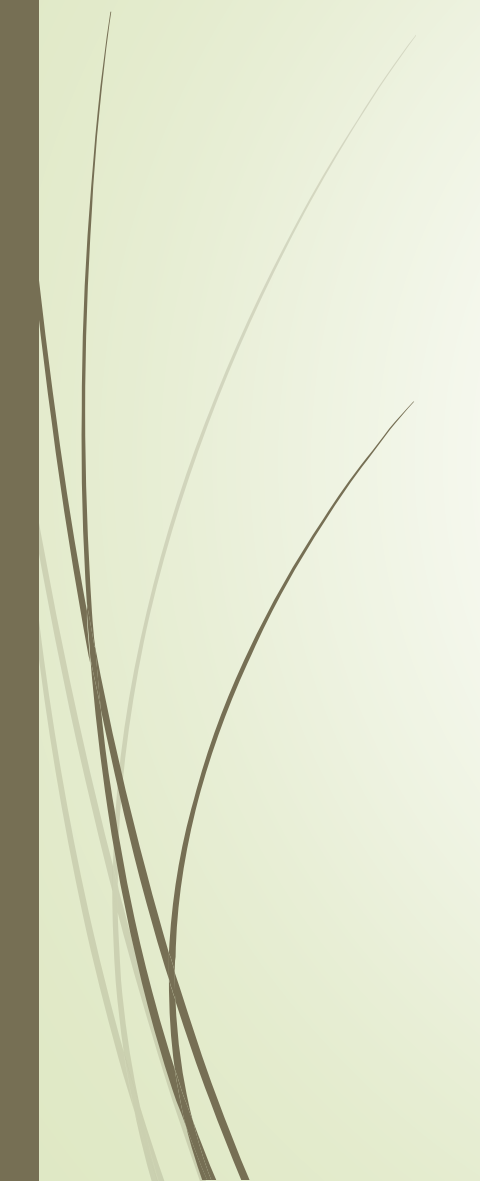




Clase N° 10



Sub-consultas

Conceptualización

- Una **subconsulta** es una sentencia SELECT que aparece **dentro** de otra sentencia SELECT (llamada **consulta principal**)

➤ Una **subconsulta** *tiene la misma sintaxis que una sentencia SELECT normal exceptuando que aparece encerrada entre paréntesis.*

- Se puede encontrar en:
 - la lista de selección,
 - en la cláusula **WHERE** o
 - en la cláusula **HAVING** de la consulta principal.

A tener en cuenta...

Cuando se ejecuta una consulta que contiene una subconsulta, **ésta se ejecuta por cada fila de la consulta principal.**

Se aconseja **no utilizar campos calculados** en las subconsultas, ya que la ralentizan.

Las consultas que utilizan subconsultas suelen ser **más fáciles de interpretar por el usuario**

Estas subconsultas permiten realizar operaciones avanzadas como filtros, cálculos y comparaciones basadas en otros datos

Conceptualización amplia

- Una subconsulta es una instrucción SELECT **anidada** dentro de una instrucción

- ☐ SELECT,

- ☐ INSERT...INTO,

- ☐ DELETE, o

- ☐ UPDATE,

- Se pueden utilizar tres formas de sintaxis para crear una:

- ❖ comparación [ANY | ALL | SOME] (instrucc. Sql)
- ❖ expresión [NOT] IN (instrucción sql)
- ❖ [NOT] EXISTS (instrucción sql)

- ❖ **comparación** [ANY | ALL | SOME] (instrucc. Sql)
- ❖ expresión [NOT] IN (**instrucción sql**)
- ❖ [NOT] EXISTS (**instrucción sql**)

Compara- ción:

- Es una expresión y un operador de comparación que compara la expresión con el resultado de la subconsulta

Expres- sión:

- Es una expresión por la que se busca el conjunto resultante de la subconsulta

Instrucción sql:

- Es una instrucción SELECT, que sigue el mismo formato y reglas que cualquier otra instrucción SELECT. (Debe ir entre paréntesis)

Tipos de Subconsultas

Subconsultas Escalares: Devuelven un **único valor** (una sola fila y columna), que puede utilizarse en expresiones de la consulta externa

Subconsultas de Fila Única: Devuelven **una sola fila** de datos, pero pueden tener varias columnas. Se usan cuando se necesita un conjunto específico de valores.

Subconsultas de Múltiples Filas: Devuelven **varias filas** de una sola columna y, generalmente, se utilizan con operadores como IN, ANY, o ALL.

Subconsultas Correlacionadas: Se ejecutan una vez por cada fila de la consulta externa, ya que ***hacen referencia a las columnas de la consulta principal.***

Ejemplos escalares

- *Averiguar qué empleado tiene el sueldo más alto*

```
SELECT empleado, sueldo
FROM Empleados
WHERE sueldo = (SELECT max(sueldo)
                FROM Empleados)
```

(SELECT max(sueldo) FROM empleados)

es una subconsulta escalar que devuelve el sueldo máximo **(un solo resultado)**.

La consulta externa selecciona a los empleados cuyo sueldo es igual al máximo.

Ejemplos escalares

- *Encontrar a los empleados cuyo sueldo es mayor al sueldo promedio.*

SELECT empleado, sueldo

FROM empleados

WHERE sueldo > (**SELECT AVG(sueldo) FROM empleados**);

(SELECT AVG(sueldo) FROM empleados)

es una subconsulta escalar que devuelve el sueldo promedio (**un solo resultado**).

La consulta externa selecciona a los empleados cuyo sueldo es mayor que el promedio

Ejemplo de múltiples filas

- *Encontrar los productos vendidos con un descuento igual o mayor al 25 por ciento:*

SELECT *

FROM Productos

WHERE IDProducto **IN**

*(**SELECT** IDProducto*

***FROM** DetalladePedidos*

***WHERE** Descuento >= 0.25);*

Ejemplo de múltiples filas

- *Mostrar los empleados no trabajan en ventas*

SELECT empleado, sueldo

FROM empleados

WHERE id_dpto **IN**

*(**SELECT** id_dpto **FROM** dpto **WHERE** dpto <> 'ventas');*

*(**SELECT** id_dpto **FROM** dpto **WHERE** dpto <> 'ventas')*

devuelve los códigos de departamentos que no son ventas.

La consulta externa selecciona a los empleados en esos departamentos.

Supongamos utilizar un signo igual, la consulta daría error si hubiera más de un departamento que no sea ventas, debido a que trata de igualar un valor (único) con un conjunto de valores.

Ejemplos de múltiples filas

- *Se requiere saber si **ALGUN** precio de los libros de "Borges" es mayor a **ALGUN** precio de los libros de "Richard Bach":*

```
SELECT titulo, precio  
FROM libros  
WHERE autor='Borges' and  
precio > ANY (SELECT precio FROM libros WHERE autor='Bach')
```

El precio de cada libro de "Borges" es comparado con cada valor de la lista de valores retornada por la subconsulta; si **ALGUNO** cumple la condición, es decir, es mayor a **ALGUN** precio de "Richard Bach", se lista

Ejemplos de múltiples filas

- *Se requiere saber si ALGUN precio de los libros de "Borges" es mayor a TODOS los precio de libros de "Richard Bach":*

```
SELECT titulo, precio
FROM articulo INNER JOIN autor au USING(id_autor)
WHERE autor like '%Borges' and precio > ALL
    (SELECT precio FROM articulo WHERE autor like '%Bach')
```

El precio de cada libro de "Borges" es comparado con cada valor de la lista de valores retornada por la subconsulta; si cumple la condición, es decir, si es mayor a TODOS los precios de "Richard Bach" (o al mayor), se lista

Ejemplos de múltiples filas

- *Se requiere saber si TODAS las editoriales que publicaron libros de "Borges" coinciden con TODAS las editoriales que publicaron libros de "Richard Bach"*

```
SELECT titulo FROM articulo inner join autor AU using (id_autor)
WHERE autor like '%Borges%' and id_editorial = ALL
    (SELECT ART.id_editorial
     FROM editorial ED INNER JOIN articulo ART
     USING(id_editorial)
     WHERE AU.autor like '%Richard Bach%')
```

La consulta interna (subconsulta) retorna una lista de valores de un solo campo (puede ejecutar la subconsulta como una consulta para probarla), luego, la consulta externa compara cada valor de "**codigoeditorial**" con cada valor de la lista, si TODOS coinciden, devuelve los títulos

Subconsulta de una columna que retorna múltiples filas

Como indica su nombre, la subconsulta retornará varias filas, por ende ya **no podemos usar operadores binarios** para la condición del WHERE.

En estos casos, se debe usar operadores para múltiples resultados como **ALL, IN, ANY**, etc.

Ejemplos Correlacionadas

- *Buscar los empleados cuyo sueldo es superior al sueldo promedio en su propio sector.*

```
SELECT empleado, sueldo
FROM empleados e1
WHERE sueldo > (SELECT AVG(sueldo) FROM
empleados e2
WHERE e2.idsector = e1.idsector);
```

```
SELECT AVG(salario) FROM empleados e2 WHERE e2.id_depto = e1.id_depto)
```

se ejecuta para cada empleado, cada empleado se compara con el sueldo promedio de su sector.

Referencias Externas

A menudo, es necesario, dentro del cuerpo de una subconsulta, hacer referencia al **valor de una columna en la fila actual de la consulta principal**, ese nombre de columna se denomina referencia externa

Una **referencia externa** es un nombre de columna que estando en la subconsulta, no se refiere a ninguna columna de las tablas designadas en la **FROM** de la subconsulta sino a una **columna de las tablas designadas en la FROM de la consulta principal**.

Como la subconsulta se ejecuta por **cada fila** de la consulta principal, **el valor de la referencia externa irá cambiando**

Ejemplos

```
SELECT idempleado, empleado, (SELECT  
    min(fechapedido) as PrimerPEdido  
    FROM pedidos PE  
    WHERE PE.idempleado = idempleado)  
FROM empleados
```

- En este ejemplo la consulta principal es **SELECT... FROM empleados**.
- La subconsulta es **(SELECT MIN(fechapedido) FROM pedidos PE WHERE PE.idempleado = idempleado)**
Y es parte de la lista de campos

En esta subconsulta tenemos una referencia externa: (id_emp) que es un campo de la tabla empleados (origen de la consulta principal).

Referencias Externas: Ejemplos

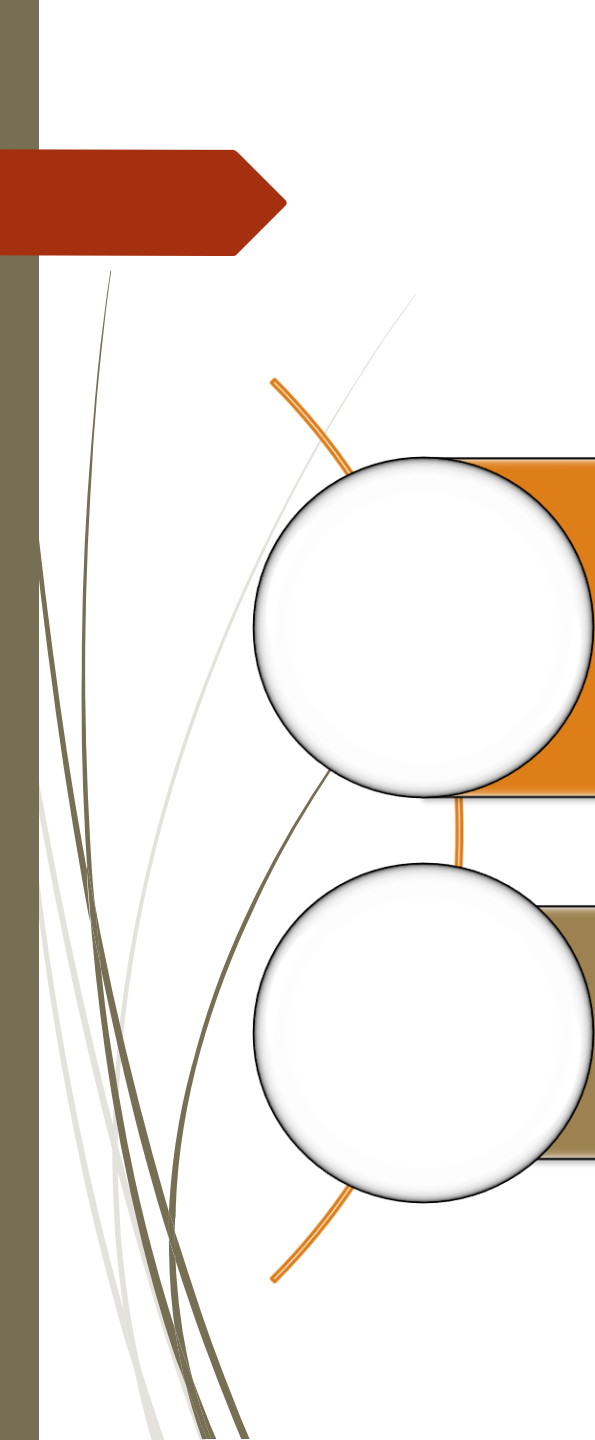
➤ **¿Qué pasa cuando se ejecuta la consulta principal?**

- se toma el primer registro y se calcula la subconsulta sustituyendo idempleado por el valor que tiene en el primer empleado. La subconsulta obtiene la fecha más antigua en los pedidos, por ejemplo, del rep = 101,
- se continua con el segundo registro y se calcula la subconsulta con idempleado = 102 (id_emp del segundo registro)... y así sucesivamente hasta llegar al último.

➤ **Al final obtenemos una lista con el número, nombre y fecha del primer pedido de cada empleado.**

Recordemos → como la subconsulta se ejecuta por cada fila de la consulta principal, **el valor de la ref. externa va cambiando.**

Reglas de las Subconsultas



La lista de selección de una subconsulta que se especifica con un **operador de comparación**, sólo puede incluir un nombre de expresión o columna (excepto **EXISTS** e **IN**, que operan en **SELECT *** o en una lista respectivamente).

La palabra clave **DISTINCT** **no se puede usar** con subconsultas que incluyan **GROUP BY**.

Reglas de las Subconsultas (2)

Puesto que deben devolver un solo valor, las subconsultas que se especifican con un **operador de comparación sin modificar** (no seguido de la palabra clave ANY o ALL) no pueden incluir las cláusulas GROUP BY y HAVING.

La lista de selección de una sub-consulta especificada con **EXISTS**, **por convención**, tiene un **asterisco (*)** en lugar de un nombre de columna.

Las reglas de una subconsulta especificada con EXISTS son idénticas a las de una lista de selección estándar, porque este tipo de subconsulta crea una prueba de existencia y **devuelve TRUE o FALSE** en lugar de datos.

Predicado **ANY**, **ALL**.

Test de Comparación cuantificada

- Se puede utilizar el predicado **ANY** (o **SOME**), para recuperar registros de la consulta principal, que satisfagan la comparación con cualquier otro registro recuperado en la subconsulta.

```
SELECT * FROM Productos PR
WHERE PR.Precio > ANY
    (SELECT PROD.Precio
     FROM DetalledePedidos INNER JOIN
        Productos PROD USING (idproducto)
     WHERE Descuento >= 0.25);
```

- ❑ devuelve todos los productos cuyo precio es mayor que el de **cualquier** producto vendido con un descuento igual o mayor al 25%:

Predicado **ANY**, **ALL** ₂

ALL Predicado **ALL** se utiliza para recuperar **única-mente** aquellos registros de la consulta principal que satisfacen la comparación con **todos los registros recuperados** en la subconsulta.

ANY Si se utiliza **ANY** en lugar de **ALL** en el ejemplo anterior, la consulta devolverá aquellos productos cuyo precio sea mayor **que el de alguno de los productos** vendidos con un descuento igual o mayor al 25%.

Diferencia **ANY** esto es mucho menos restrictivo que **ALL**

Predicado **IN** / **NOT IN**. Test de pertenencia a un conjunto

- El predicado **IN** se emplea para recuperar únicamente aquellos registros de la consulta principal para los que algunos registros de la subconsulta contienen un valor igual.
- ❑ *El ejemplo siguiente muestra todos los productos y su precio que han sido pedidos en una cantidad menor a 5.*

```
SELECT producto, precio FROM Productos
WHERE idproducto IN (SELECT idproducto
                        FROM DetalledePedidos
                        WHERE cantidad < 5);
```

Predicado **IN** / **NOT IN**. Test de pertenencia a un conjunto

Inversamente, se utiliza **NOT IN** para recuperar únicamente aquellos registros de la consulta principal para los que **no hay ningún registro** de la subconsulta que contenga un valor igual.

```
SELECT idcliente, cliente  
FROM Clientes  
WHERE idcliente NOT IN  
      (SELECT idcliente FROM Pedidos);
```


Predicado **IN** / **NOT IN**. Test de pertenencia a un conjunto

- Hay que tener especial cuidado con los valores nulos cuando utilizamos el operador **NOT IN** porque el resultado obtenido no siempre será el deseado. Cuando la subconsulta devuelve algún nulo el resultado es falso o nulo pero nunca verdadero. **EJEMPLOS:**

```
SELECT oficina, ciudad  
FROM oficinas  
WHERE oficina IN (SELECT oficina  
FROM empleados)
```

Obtenemos las oficinas que tienen algún empleado.

```
SELECT oficina, ciudad  
FROM oficinas  
WHERE oficina NOT IN  
(SELECT oficina FROM  
empleados)
```

Al contrario de lo que se puede pensar **esta consulta NO devuelve las oficinas que no tienen ningún empleado** porque la subconsulta devuelve algún **NULL** (por los empleados que no están asociados a ninguna oficina).

```
SELECT oficina, ciudad  
FROM oficinas  
WHERE oficina NOT IN (SELECT  
oficina FROM empleados WHERE  
oficina IS NOT NULL)
```

Así sí.



A tener en cuenta...

La expresión "**= ANY**" equivale a "**IN**".

La expresión "**<> ALL**" equivale a "**NOT IN**".

Predicado EXISTS. Test de existencia

El predicado **EXISTS** / **NOT EXISTS** se utiliza en *comparaciones de verdadero/falso para determinar si la subconsulta devuelve algún registro.*

Se puede utilizar también alias del nombre de la tabla en una subconsulta para referirse a tablas listadas en la cláusula **FROM** fuera de la subconsulta.

El *ejemplo* siguiente *devuelve los productos, con su respectivo precio que tuvieron algún pedido realizado por una cantidad menor a 5.* (consulta ya realizada con **IN**)

Cuando se trabaja con tablas muy voluminosas el test **EXISTS** suele dar mejor rendimiento que el test **IN**.

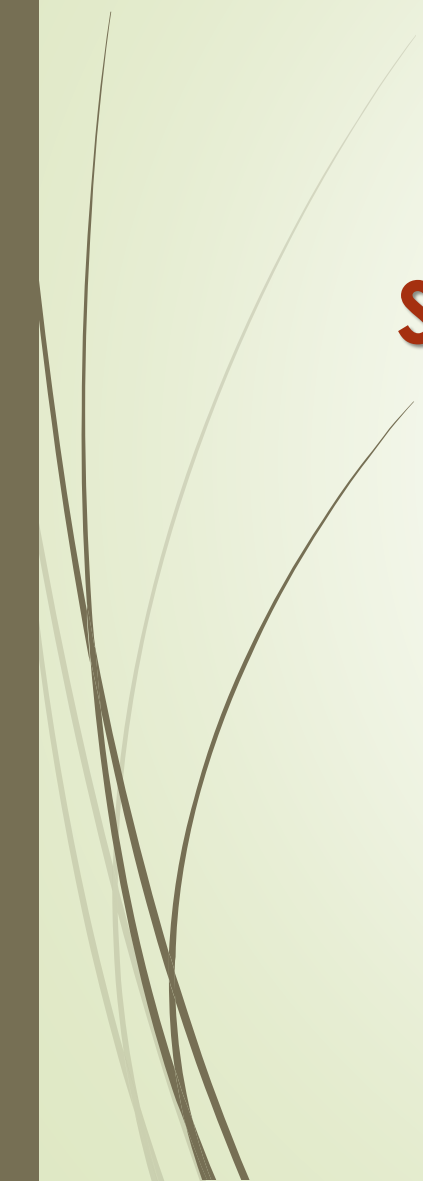
Predicado **EXISTS**: ejemplo

```
SELECT producto,precio FROM Productos  
WHERE EXISTS
```

```
(SELECT * FROM detalladepedidos DP WHERE  
productos.idproducto=DP.idproducto and  
cantidad<5)
```

Este ejemplo obtiene lo mismo que el ejemplo del test **IN**. Observa que delante de **EXISTS** no va ningún nombre de columna.

Hemos añadido una condición adicional al **WHERE**, la de la referencia externa.



Subconsultas con comandos de manipulación de datos como **UPDATE y DELETE**

Actualizar datos usando subconsultas

Para actualizar datos en función de valores en otras tablas o basados en alguna consulta compleja, se usa una subconsulta en la cláusula SET o WHERE.

- ***Aumentar la categoría de los clientes que tengan al menos un pedido realizado en el año 2023***

```
UPDATE clientes SET categoría = categoría + 1
WHERE idcliente IN (SELECT idcliente
                     FROM pedido
                     WHERE año = 2023)
```

Este comando actualizará la categoría de los clientes que hicieron pedidos en 2023, sumándole 1 a la categoría actual.

Actualizar datos usando subconsultas

- **Actualizar el precio de cada producto para que sea el precio máximo de los productos dentro de su misma categoría**

Suponga la Tabla: **Producto**(id_producto, nombre, precio, id_categoría)

```
UPDATE producto
```

```
SET precio = (SELECT MAX(precio) FROM productos AS p  
WHERE p.id_categoria = productos.id_categoria)
```

La subconsulta devuelve el precio máximo dentro de cada categoría, actualizando así todos los productos al precio máximo de su categoría.

Actualizar datos usando subconsultas

- **Actualizar el sueldo de los empleados que cobran menos del promedio de su departamento a dicho valor.**

Suponga la Tabla: **Empleado**(id_empleado, nombre, sueldo, id_depto)

UPDATE empleado

SET sueldo = (**SELECT AVG**(sueldo) **FROM** empleado **AS** e
WHERE e.id_depto = empleado.id_depto)

WHERE sueldo < (**SELECT AVG**(sueldo) **FROM** empleado **AS** e
WHERE e.id_depto = empleado.id_depto)

La consulta actualiza el sueldo de los empleados al sueldo promedio de los empleados del departamento, en caso que ganaran menos que el sueldo promedio.

Eliminar datos usando subconsultas

Para eliminar datos usando subconsultas; se utiliza la cláusula WHERE

- ***Eliminar a todos los clientes que no tienen ningún pedido asociado***

DELETE FROM cliente

WHERE id_cliente **NOT IN** (*SELECT id_cliente FROM pedido*)

Elimina todos los clientes cuyo id_cliente no se encuentra en la tabla pedido.

Eliminar datos usando subconsultas

Eliminar clientes sin pedidos recientes (en el último año)

Suponga las tablas:

Cliente(id_cliente, nombre)

Pedido(id_pedido, id_cliente, fecha_pedido)

DELETE FROM cliente

WHERE id_cliente **NOT IN** (*SELECT id_cliente*

FROM pedido

WHERE fecha_pedido >= '2023-01-01')

Elimina los clientes cuyo id_cliente no aparece en la subconsulta (que contiene los clientes que sí tienen pedidos desde el 1 de enero de 2023)

Eliminar datos usando subconsultas

Eliminar productos que nunca fueron comprados

Suponga las tablas:

Producto(idproducto, nombre, precio)

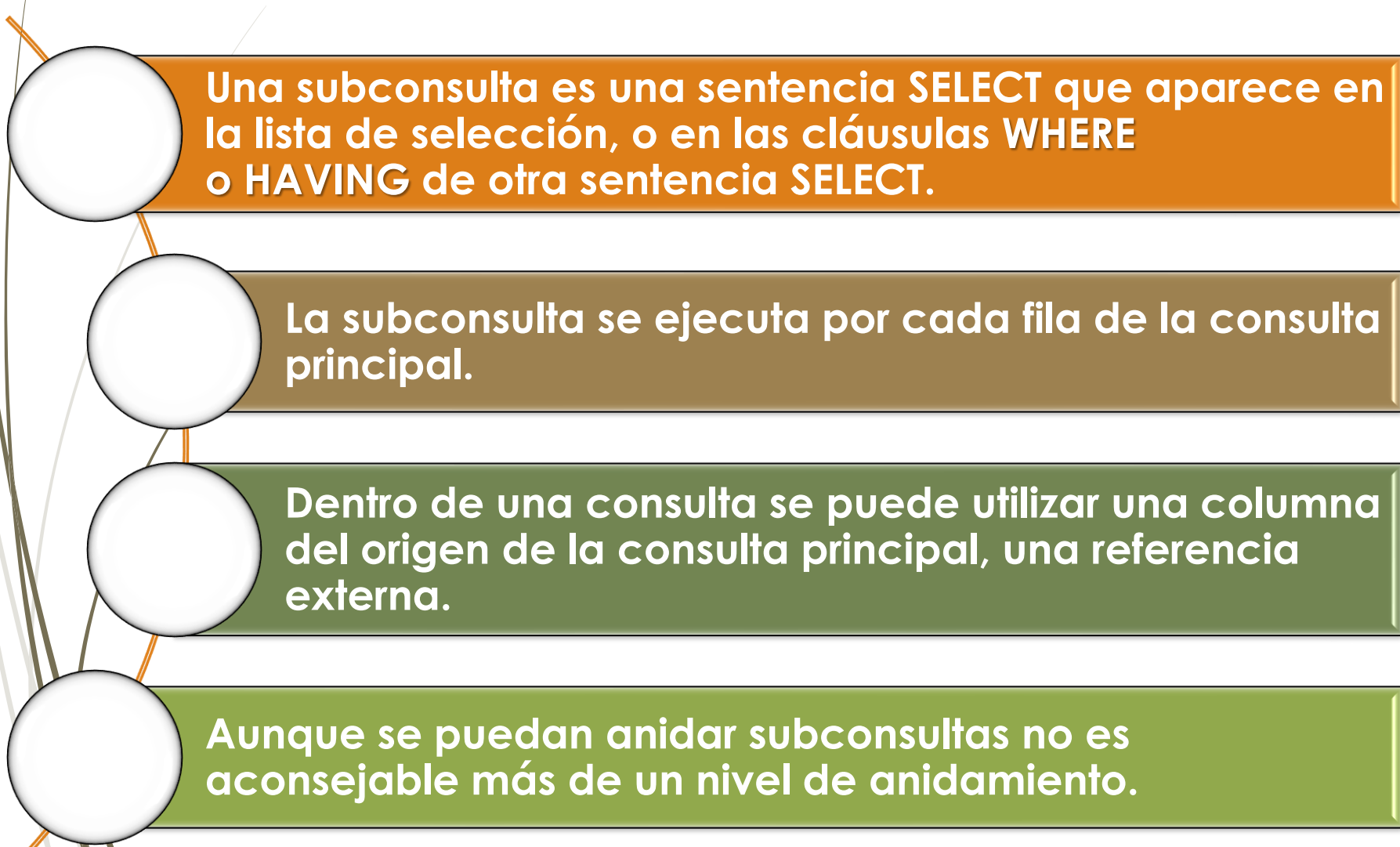
Venta(idventa, idproducto, cantidad)

DELETE FROM producto

WHERE idproducto **NOT IN** (***SELECT DISTINCT** idproducto
FROM Venta*)

Elimina los productos cuyo id_producto no aparece en la tabla ventas. El uso de DISTINCT es opcional, pero ayuda a evitar duplicados en la subconsulta, lo que mejora el rendimiento.

Resumen



Una subconsulta es una sentencia SELECT que aparece en la lista de selección, o en las cláusulas WHERE o HAVING de otra sentencia SELECT.

La subconsulta se ejecuta por cada fila de la consulta principal.

Dentro de una consulta se puede utilizar una columna del origen de la consulta principal, una referencia externa.

Aunque se puedan anidar subconsultas no es aconsejable más de un nivel de anidamiento.

- 
- A veces las **subconsultas** son la forma más cómoda de escribir una consulta **compleja**,
 - y otras veces...

son la única manera de lograrlas.

Optimización de Subconsultas

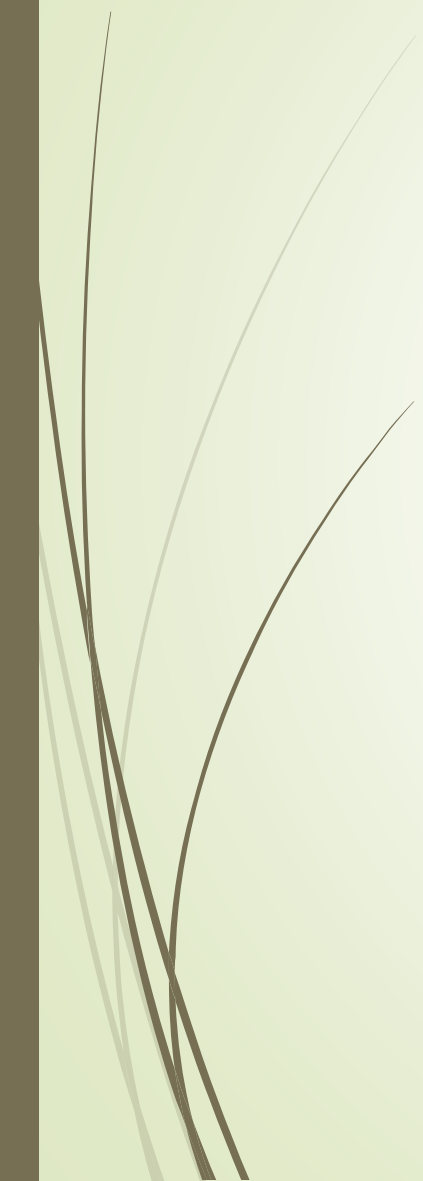
- Es importante conocer alternativas para mejorar la performance de una consulta.

► ***SELECT DISTINCT t1.column1 FROM t1, t2 WHERE t1.column1 = t2.column1;***

Reemplace por:

► ***SELECT DISTINCT column1 FROM t1 WHERE t1.column1 IN (SELECT column1 FROM t2);***

► ***<http://mysql.localhost.net.ar/doc/refman/5.0/es/optimizing-subqueries.html>***



Las **VISTAS** en SQL

Las VISTAS en SQL

Una **vista** se especifica a través de una expresión de consulta (una sentencia SELECT) que la calcula y que puede realizarse sobre una o más tablas

CREATE [OR REPLACE] VIEW *name* AS “sentencia SQL”

Ejemplo: **Listado de la suma de sueldos por cargo**

CREATE OR REPLACE VIEW *SumSuelCargo* as

SELECT cargo, sum(sueldo) AS TotalSueldos

FROM empleados INNER JOIN Cargos

USING(idcargo) GROUP BY cargo

SELECT * FROM *SumSuelCargo*

Sintaxis

CREATE VIEW nombre_table_virtual **as** "sentencia SQL"

Ejemplo: **Listado de sueldo promedio por cargo**

Create view **SuelPromCargo** as

```
SELECT cargo, avg(sueldo) as SueldoPromedio  
FROM empleados INNER JOIN Cargos  
USING(idcargo) GROUP BY cargo
```

SELECT * from SuelPromCargo

Vistas en SQL

Una **vista** es una consulta que se presenta como una **tabla (virtual)** a partir de un conjunto de tablas en una base de datos relacional.

Las **vistas** tienen la **misma estructura** que una tabla: filas y columnas. La única diferencia es: sólo se **almacena** de ellas la **definición**, **no** datos.

Simplificación: oculta la complejidad a los usuarios, proporciona acceso más sencillo a datos específicos.

Seguridad: limita el acceso a ciertas columnas o datos sensibles

Mantenimiento de Integridad: Permite acceder a datos actualizados sin duplicarlos, lo que ayuda a evitar inconsistencias.

Características

Datos

- No contiene datos propios, estos provienen de otras tablas con datos reales

Solo lectura

- No permite la inserción, actualización y eliminación de datos, solo la lectura

Uso

- Su utilización es como el de una tabla, podemos usar cualquier sentencia de tipo SELECT sobre ellas.

Aplicación

El uso de las vistas ofrece una infinidad de aplicaciones.



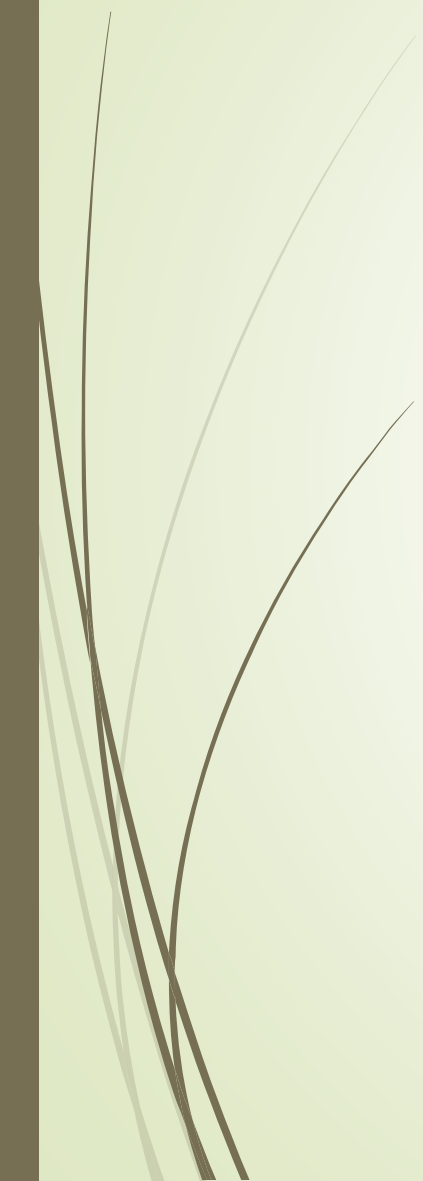
Ventajas

Facilita el manejo de grandes volúmenes de información, haciéndolos mas fáciles y rápidos de manipular

Brinda mayor seguridad a la información.

Evita la redundancia de la información

Mejora la metodología de trabajo, se hace mas organizada



Muchas gracias